IA

INVOICE AGENT

Complete Web Application Guide

Setup, run, use, test, and troubleshoot the Next.js and FastAPI invoice checking application.

FRONTEND	BACKEND	AI WORKFLOW
Next.js 16 TypeScript Tailwind CSS	FastAPI PDF validation Safe JSON API	LangGraph Structured extraction Deterministic checks

Version 1.0

Repository: souravhub/invoice-agent

Prepared: 7 October 2026

01 Purpose and scope

Invoice Agent is a local web application for uploading one text-based invoice PDF, extracting structured invoice data with an AI provider, applying deterministic checks, and presenting either **Passed configured checks** or **Needs review** with reasons.

Important decision boundary

A passing result is not payment authorization and is not proof that the invoice or vendor is authentic. The application checks extraction completeness and arithmetic consistency only.

Included in this version

- Responsive upload page for one PDF at a time.
- Vendor, invoice number, date, PO number, currency, amounts, and line-item display.
- JSON result download and clear browser error messages.
- 20 MB server-side upload limit, PDF content validation, and text extraction checks.
- Existing LangGraph extraction, retry, validation, and escalation behavior.
- OpenAI or Google Gemini selected through backend environment settings.

Not included

- Database, user accounts, payment integration, deployment, or OCR.
- Image-only scans are rejected with an OCR-required message.
- No normal user CLI is required; the application is used in the browser.

Technology summary

Layer	Technology	Responsibility
Browser	Next.js, TypeScript, Tailwind CSS	Upload, status, fields, line items, errors, JSON download
API	Python, FastAPI	Validation, limits, CORS, temporary files, safe errors
Agent	LangGraph, LangChain, Pydantic	Extraction, retry routing, checks, review decision
PDF	pdfplumber	Reads embedded text; OCR is intentionally excluded

02 Project structure and request flow

```
invoice-agent/
|-- frontend/
| |-- app/ Next.js pages and styles
| |-- lib/api.ts Browser API client and types
| |-- .env.example Public backend URL placeholder
| `-- package.json
|-- backend/
| |-- main.py FastAPI routes, CORS, uploads
| |-- service.py PDF/config validation and graph adapter
| |-- schemas.py Documented API response models
| |-- agent/
| | |-- workflow.py LangGraph pipeline and review rules
| | `-- schema.py Invoice and line-item models
| |-- tests/ API and rule tests
| |-- requirements.txt
| `-- .env.example Private backend settings template
|-- invoices/ Generated test PDFs
|-- answer_keys/ Expected extraction results
|-- reports/ Preserved evaluation reports
`-- notes.md Project history and rule rationale
```

End-to-end request flow

Step	Component	What happens
1	Browser	User chooses one PDF and clicks Check invoice.
2	Frontend	Sends multipart field named file to POST /api/invoices/check.
3	FastAPI	Checks MIME type, 20 MB limit, PDF signature, and extractable text.
4	Worker thread	Runs build_graph().invoke() without blocking the event loop.
5	Agent	Extracts fields, retries missing extraction data, validates, and decides.
6	FastAPI	Returns the documented JSON response and deletes temporary files.
7	Frontend	Displays details and enables complete JSON download.

Temporary-file isolation

Each upload is written as invoice.pdf inside a unique temporary directory. The directory is removed after processing on success, validation failure, configuration failure, or provider failure.

03 First-time setup on Windows

Prerequisites

- Python 3.12 or newer.
- Node.js 20.9 or newer; Node.js 22 LTS is recommended.
- An OpenAI or Google Gemini API key.

Open PowerShell in the repository

```
cd D:\Trigital\invoice-agent
```

Install backend dependencies

```
python -m venv venv
.\venv\Scripts\python.exe -m pip install --upgrade pip
.\venv\Scripts\python.exe -m pip install -r backend\requirements.txt
```

Install frontend dependencies

```
npm --prefix frontend install
```

Create local configuration files

```
Copy-Item backend\.env.example backend\.env
Copy-Item frontend\.env.example frontend\.env.local
```

If backend/.env already exists

Do not overwrite it. Open the existing file and verify the provider, model, key, and CORS origin instead.

Backend environment

```
AI_PROVIDER=openai
AI_MODEL=gpt-4o-mini
OPENAI_API_KEY=replace-with-your-real-key
GEMINI_API_KEY=replace-with-your-real-key
CORS_ORIGINS=http://localhost:3000
```

For Gemini, set AI_PROVIDER=gemini, choose a supported Gemini model, and provide GEMINI_API_KEY. Real keys belong only in backend/.env. Actual .env files are ignored by Git.

Frontend environment

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000
```

04 Run and use the web application

Terminal 1 - start FastAPI

```
cd D:\Trigital\invoice-agent
.\venv\Scripts\python.exe -m uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000
```

Terminal 2 - start Next.js

```
cd D:\Trigital\invoice-agent
npm --prefix frontend run dev
```

Open these URLs

URL	Purpose	Expected result
http://localhost:3000	Web application	Invoice upload screen
http://localhost:8000/api/health	Backend health	{"status":"ok"}
http://localhost:8000/docs	Interactive API docs	FastAPI Swagger UI

Normal user workflow

- Click Browse and select one text-based PDF.
- Click Check invoice. A spinner appears while the agent runs.
- Review the status, reasons, invoice fields, amounts, and line items.
- Use Download JSON to save the complete API result.
- Selecting another file clears the previous result and error message.

Keep both terminals open

The browser cannot check an invoice if FastAPI is stopped. If the page says it cannot reach the invoice service, test the health URL first and restart Terminal 1 if needed.

Stop the servers

Press Ctrl+C once in each terminal.

05 API contract

GET /api/health

Checks only that the FastAPI process is available. It does not call the AI provider.

```
{"status":"ok"}
```

POST /api/invoices/check

Send multipart/form-data containing one field named file. The file must declare application/pdf, contain a PDF signature, be at most 20 MB, and contain extractable text.

Successful response

```
{
"invoice": {
"vendor_name": "Example Vendor",
"invoice_number": "INV-001",
"invoice_date": "2026-10-07",
"po_number": "PO-100",
"currency": "USD",
"line_items": [
{
"description": "Consulting service",
"quantity": 1,
"unit_price": 100,
"amount": 100
}
],
"subtotal": 100,
"discount": null,
"shipping": null,
"tax": 10,
"total": 110
},
"status": "auto_approved",
"reasons": [],
"attempts": 1
}
```

The API retains the internal auto_approved value for evaluation compatibility. The browser presents this as Passed configured checks.

06 Agent behavior and configured checks

```
PDF text -> AI extraction -> deterministic validation -> decision
^ |
`-- retry missing extraction data only
```

Extraction retries

The agent can retry when the AI misses the vendor, total, or all line items. MAX_RETRIES is 2, allowing up to three total extraction attempts.

Arithmetic is never sent back to the AI

Printed arithmetic mismatches are facts about the document. They are reported for human review instead of asking the AI to rewrite values until the sums close.

Rule	Check	Result when triggered
Required fields	Vendor, invoice date, and total are present	Needs review
Line items	At least one line item was extracted	Needs review
Currency	A currency code was extracted	Needs review
Subtotal	Sum of items minus discount equals subtotal within 0.01	Needs review
Total	Subtotal plus tax plus shipping equals total within 0.01	Needs review
Retries	Validation still fails after retries	Needs review
Changed answer	More than one AI attempt was used	Needs review

Status interpretation

API value	Browser label	Meaning
auto_approved	Passed configured checks	No configured extraction or arithmetic rule triggered.
needs_review	Needs review	One or more reasons explain what a person must inspect.

Always apply business controls

Neither status verifies vendor identity, detects fraud, confirms goods were received, checks duplicate payment, or authorizes payment.

07 Validation, security, and error handling

Upload controls

- The frontend accepts one PDF and performs an early 20 MB check.
- The backend independently enforces the 20 MB limit.
- The backend checks application/pdf and searches the first 1,024 bytes for %PDF-.
- pdfplumber confirms the file is parseable and contains at least one page with text.

Safe failure responses

HTTP	Code	When used
400	invalid_pdf	Empty, spoofed, corrupt, encrypted, or otherwise unreadable PDF
413	file_too_large	Upload exceeds 20 MB
415	pdf_required	Declared content type is not application/pdf
422	unreadable_pdf	PDF contains no extractable text and needs OCR
502	ai_provider_error	Provider could not process the invoice
503	configuration_error	Provider, model, or required key is missing/invalid

Secret handling

- API keys, AI_PROVIDER, AI_MODEL, and CORS_ORIGINS stay in backend/.env.
- The frontend receives only NEXT_PUBLIC_API_BASE_URL; NEXT_PUBLIC values are browser-visible.
- Actual .env files, node_modules, build output, caches, and results are ignored by Git.
- Provider exceptions and internal stack traces are not returned to the browser.

CORS

The default allowed origin is http://localhost:3000. For another frontend address, update CORS_ORIGINS in backend/.env and restart FastAPI.

08 Testing and verification

Backend tests

```
.\venv\Scripts\python.exe -m pytest backend\tests -q
```

The automated suite covers health, documented response shape, content-type validation, PDF signature checks, upload size limits, scanned/textless PDFs, missing configuration, provider failure sanitization, temporary-directory cleanup, arithmetic, and retry escalation.

Mocked AI coverage

API success and provider-failure tests use mocked AI processing, so tests do not spend provider quota. Deterministic rule tests use real Invoice and LineItem models.

Frontend checks

```
npm --prefix frontend run typecheck
npm --prefix frontend run build
```

Manual smoke test

- Open /api/health and confirm a 200 response.
- Open the web app and select invoices/invoice_01.pdf.
- Confirm loading feedback appears while processing.
- Confirm fields, amounts, status, and line items appear.
- Download JSON and confirm it includes invoice, status, reasons, and attempts.
- Select another PDF and confirm the previous result clears immediately.

Evaluation utility

The repository retains backend/evaluate.py for development evaluation against answer_keys/. It is not required to use the web application and may make many paid or quota-limited AI calls.

```
.\venv\Scripts\python.exe -m backend.evaluate
.\venv\Scripts\python.exe -m backend.evaluate --reuse
```

09 Troubleshooting

Symptom	Likely cause	Resolution
Could not reach the invoice service	FastAPI is stopped or frontend URL is wrong	Open /api/health. Start FastAPI on port 8000. Verify NEXT_PUBLIC_API_BASE_URL, then restart Next.js after env changes.
Health URL does not open	Nothing listens on port 8000	Run the backend command in a separate terminal and keep it open. Read the terminal error if it exits.
Configuration error	Key/provider/model missing	Check backend/.env. Set the selected provider's key. Restart FastAPI.
AI provider error	Quota, billing, network, model, or provider outage	Check provider account and model availability. Retry later. Server logs intentionally remain more detailed than browser errors.
No readable text / OCR required	Image-only scanned PDF	Use a text-searchable PDF. OCR is outside this version.
PDF required	Wrong MIME type	Choose a real PDF through the Browse control; do not rename another format to .pdf.
File too large	PDF exceeds 20 MB	Reduce or split the document before upload.
CORS error	Frontend origin differs from allowed origin	Add the exact origin to CORS_ORIGINS and restart FastAPI.
Port already in use	Another process owns 3000 or 8000	Stop the other process or use another port and update both environment settings.

Quick diagnostic commands

```
Invoke-RestMethod http://localhost:8000/api/health
Get-NetTCPConnection -LocalPort 8000 -State Listen
Get-NetTCPConnection -LocalPort 3000 -State Listen
```

Operational checklist

- Keep backend/.env private and out of screenshots, chat, and source control.
- Review Needs review reasons before using extracted values.
- Apply separate vendor, duplicate, approval, and payment controls.
- Do not treat a passing configured-check result as authenticity verification.
- Stop both development servers with Ctrl+C when finished.

Ready state

The system is ready when both servers stay running, /api/health returns {"status":"ok"}, the browser opens on port 3000, and a text-based test invoice returns structured results.